Evaluación Técnica para Desarrolladores Frontend, Backend y Fullstack

Marzo, 2026

Control de Revisión

Versión	Descripción del Cambio	Realizado por	Revisado por	Aprobado por	Fecha
v.1	Generación de documento	Miguel Lozano	Luis Miguel Alvear	Luis Miguel Alvear	02/03/2026

CONFIDENCIALIDAD

Toda la información contenida en este documento pertenece a SERVICIOS INFORMATICOS INTEGRALES LIBELULASOFT Cía. Ltda, y no podrán ser modificado, duplicado, publicado o distribuido en forma alguna, ya sea parcial o total, sin previa autorización.

Evaluación Técnica para Desarrolladores Frontend, Backend y Fullstack

---

Objetivo:

El objetivo de esta prueba es evaluar tu capacidad para diseñar e implementar una solución funcional basada en un caso de negocio real.

La prueba consiste en desarrollar una aplicación que implemente el flujo Quote & Bind (cotización y emisión de pólizas) utilizando un stack moderno de frontend y backend.

La evaluación se centrará en:

- Claridad de diseño y arquitectura
- Calidad del código
- Uso correcto de API REST
- Seguridad básica (autenticación y autorización)
- Validación de datos
- Manejo consistente de errores

Descripción del Proyecto:

Desarrollar una plataforma insurtech que permita a usuarios solicitar cotizaciones de seguros y, si están conformes, confirmar la cotización para emitir una póliza. Este ejercicio modela un flujo simplificado:

- Solicitar cotización (quote) con datos básicos del riesgo.
- Recibir prima estimada y un desglose simple del cálculo.
- Confirmar la cotización y emitir una póliza (policy) persistida en base de datos.

Tecnologías esperadas

La solución debe desarrollarse utilizando tecnologías modernas del ecosistema **TypeScript / Node.js / React**.
Se pueden utilizar alternativas equivalentes siempre que el postulante mantenga consistencia técnica y documente la decisión.

Backend

- Node.js
- NestJS
- TypeScript
- PostgreSQL
- ORM: Prisma
- Autenticación: JWT
- Validación de datos: class-validator o Zod
- Documentación de API: OpenAPI / Swagger
- Testing: Vitest, Supertest

Frontend

- React
- Next.js
- TypeScript
- TailwindCSS
- Formularios y validación: React Hook Form + Zod (recomendado)
- Consumo de APIs: Fetch API o Axios
- Manejo de estado: React Context o Zustand
- Testing: Vitest

Calidad de código y herramientas

- ESLint
- Prettier
- Variables de entorno mediante. env y. env.example

CI/CD
El repositorio debe incluir un pipeline básico que ejecute:

- instalación de dependencias
- lint
- tests
- build del proyecto

Alcance:

El siguiente alcance aplica a cualquiera de las rutas de entrega (Frontend, Backend o Fullstack).

Las diferencias específicas se detallan en la sección "Rutas de entrega".

1. **Estándar API REST**

Los endpoints deben seguir prácticas REST: recursos en plural, verbos HTTP correctos, códigos de estado coherentes y contratos estables. Debe existir consistencia en naming, estructura de respuestas y manejo de errores.

2. **Seguridad: autenticación y autorización**

Las acciones sensibles deben requerir autenticación mediante token (JWT recomendado o token simulado para la ruta solo Frontend). Además, debe aplicarse autorización en endpoints sensibles (mínimo: emisión y consulta de pólizas).

3. **Catálogos**

El sistema debe exponer catálogos para alimentar formularios y restringir valores permitidos. El frontend debe consumir estos catálogos (o mocks equivalentes en la ruta solo Frontend) y el backend debe validar la pertenencia a catálogos cuando reciba requests.

Rutas de entrega:

Se debe elegir una ruta de entrega. Cada ruta tiene requerimientos equivalentes en su alcance; elige la que mejor represente tu fortaleza.

1. **Ruta Frontend**
   (UI + Contrato REST + Mock API)

Entregas un frontend completo y consumes una API mock local respetando los contratos REST definidos en este documento.

Incluye:

- Flujo completo Quote & Bind en UI.
- Consumo de catálogos desde endpoints mock.
- Autenticación simulada y envío de Authorization: Bearer <token> en llamadas protegidas.

- Validaciones, estados de carga, manejo de errores y UX clara.
- Tests mínimos de UI (componentes o flujo principal).

2. Ruta Backend

(Microservicios + PostgreSQL + Seguridad)

Entregas backend completo con microservicios, persistencia, seguridad y documentación de API.

Incluye:

- Quote Service y Policy Service.
- PostgreSQL y ORM (Prisma recomendado).
- Autenticación y autorización en endpoints sensibles.
- Catálogos expuestos vía API y validación server-side.
- Documentación OpenAPI/Swagger.
- Tests mínimos de API (unitarios e integración).

3. Ruta Fullstack

(End-to-End)

Entregas frontend y backend integrados cumpliendo todo el alcance: REST, seguridad, catálogos y persistencia.

Incluye:

- Todo lo de Ruta Frontend + Ruta Backend, integrado de extremo a extremo.
- Manejo consistente de errores y estados entre UI y API.

Flujo funcional esperado

El sistema debe soportar el siguiente flujo:

- El usuario solicita una cotización ingresando:
  - tipo de seguro
  - cobertura
  - edad
  - ubicación
- El sistema calcula una prima estimada y devuelve un desglose del cálculo.
- El usuario puede confirmar la cotización.

- El sistema emite una póliza asociada a esa cotización y la persiste en base de datos.

Contratos API REST:

Los siguientes endpoints y contratos deben implementarse tal cual o con equivalentes funcionales consistentes.

- **Resumen de endpoints**

Método	Endpoint	Auth	Descripción
GET	/catalogs/insurance-types	No	Lista tipos de seguro
GET	/catalogs/coverages?insuranceType=...	No	Lista coberturas por tipo de seguro
GET	/catalogs/locations	No	Lista ubicaciones
POST	/quotes	No	Crea una cotización y devuelve prima + breakdown
GET	/quotes/{id}	No	Obtiene una cotización por id
POST	/auth/login	No	Autenticación: retorna token (JWT o simulado según ruta)
POST	/policies	Sí	Emite una póliza desde una cotización
GET	/policies/{id}	Sí	Obtiene una póliza por id

- **Catálogos**

Catálogos mínimos requeridos:

  - Tipos de seguro: AUTO, SALUD, HOGAR (puedes ajustar nombres, manteniendo consistencia).
  - Coberturas: BASICA, ESTANDAR, PREMIUM (dependiente del tipo de seguro).
  - Ubicaciones: set acotado (por ejemplo, provincias/cantones o códigos simples).

Regla: los valores enviados en POST /quotes deben existir en los catálogos definidos.

El backend debe validar esta condición y rechazar valores inválidos.

Ejemplo de respuesta - GET /catalogs/insurance-types:

```
{
  "items": [
    {
      "code": "AUTO",
      "name": "Seguro de Auto"
    },
    {
      "code": "SALUD",
      "name": "Seguro de Salud"
    },
    {
      "code": "HOGAR",
      "name": "Seguro de Hogar"
    }
  ]
}
```

- **Quote**

POST /quotes - Request:

```
{
  "insuranceType": "AUTO",
  "coverage": "PREMIUM",
  "age": 35,
  "location": "EC-AZUAY"
}
```

POST /quotes - Response (ejemplo):

```
{
  "id": "uuid",
  "status": "QUOTED",
  "inputs": {
    "insuranceType": "AUTO",
    "coverage": "PREMIUM",
    "age": 35,
    "location": "EC-AZUAY"
  },
  "estimatedPremium": 350,
  "breakdown": [
    {
      "concept": "BASE",
      "amount": 200
    },
    {
      "concept": "AGE_FACTOR",
      "amount": 60
    },
```

```
    {
      "concept": "LOCATION_FACTOR",
      "amount": 40
    },
    {
      "concept": "COVERAGE_FACTOR",
      "amount": 50
    }
  ],
  "createdAt": "ISO-8601"
}
```

GET /quotes/{id} debe retornar la misma estructura para la cotización persistida.

- **Policy**

POST /policies (protegido) - Request:

```
{
  "quoteId": "uuid"
}
```

POST /policies (protegido) - Response (ejemplo):

```
{
  "id": "uuid",
  "quoteId": "uuid",
  "status": "ACTIVE",
  "issuedAt": "ISO-8601"
}
```

GET /policies/{id} (protegido) debe retornar la póliza persistida.

- **Auth**

Implementa un endpoint de autenticación que retorne un token. Se recomienda JWT. En la Ruta Frontend, puede ser simulado, pero debe mantenerse el header Authorization: Bearer <token>.

Ejemplo - POST /auth/login (request):

```
{
  "email": "user@example.com",
  "password": "password"
}
```

Ejemplo - POST /auth/login (response):

```
{
  "accessToken": "jwt-or-simulated-token",
  "tokenType": "Bearer"
}
```

Requerimientos

- **Backend**

Arquitectura y estructura:

- Dos microservicios mínimos: Quote Service y Policy Service.
- Comunicación entre servicios: HTTP interno o mensajería simulada (documentar decisión).
- Separación de capas: controller -> service/use-case -> data access.

Persistencia y ORM:

- PostgreSQL obligatorio.
- Uso de ORM obligatorio (Prisma recomendado).
- El postulante debe diseñar el modelo de datos necesario para soportar las entidades del sistema y dejar la definición del esquema en el repositorio.
- La base de datos debe permitir persistir quotes y policies, con una regla para evitar doble emisión de póliza por quote (recomendado: constraint única por quoteId).

Seguridad y control de acceso:

- Autenticación por token (JWT recomendado).
- Autorización aplicada al menos en POST /policies y GET /policies/{id}.
- Buenas prácticas: no hardcodear secretos; usar variables de entorno y .env.example.

Validación y errores:

- DTOs y validación de entrada (ValidationPipe con whitelist/transform recomendado).
- Validación server-side de pertenencia a catálogos (incluyendo combinaciones insuranceType + coverage).
- Manejo consistente de errores en un formato único (Problem Details recomendado).

Documentación y tests:

- Swagger/OpenAPI habilitado.
- Tests mínimos: unitarios (caso core) e integración (endpoints quotes/policies).

- **Frontend**

UX y flujo:

- Pantalla/formulario de cotización con campos: tipo de seguro, cobertura, edad, ubicación.
- Cobertura dependiente del tipo de seguro (consumo de /catalogs/coverages).
- Pantalla de resultado: prima estimada y breakdown visible y entendible.
- Acción para emitir póliza con confirmación explícita y visualización de resultado (id + estado).

Catálogos:

- Consumir /catalogs/insurance-types, /catalogs/coverages, /catalogs/locations (o mocks en Ruta Frontend).
- Manejar estados de carga y error en la carga de catálogos (con opción de reintento).
- Evitar que el usuario envíe valores fuera de catálogo; validar antes de llamar a la API.

Integración REST y errores:

- Cliente HTTP con manejo consistente de errores; mapear errores del backend a mensajes claros en UI.
- Estados de carga por acción: cotizar y emitir póliza.
- Manejo de errores globales (por ejemplo, error banner) y errores por campo cuando corresponda.

Autenticación/autorización en UI:

- Login para obtener token (real en Ruta FullStack; simulado en Ruta Frontend).
- Enviar Authorization: Bearer <token> en requests protegidos.
- Restringir/ocultar acciones sensibles si no hay sesión/token (por ejemplo, bloquear emisión de póliza).

Validación, accesibilidad y tests:

- Validaciones de formulario alineadas al contrato: requeridos, rangos (age) y valores de catálogo.

- Accesibilidad básica: labels, errores visibles, navegación por teclado.
- Tests mínimos: componentes o flujo principal (cotizar -> ver resultado -> emitir).

Requerimientos no funcionales

- **Validación**

Debe existir validación en backend (si aplica) y frontend. Los mensajes deben ser claros y accionables.

- **Manejo de errores**

Se espera un formato único y consistente para errores.

- **Observabilidad**

Logging básico con contexto (servicio, endpoint y error). No incluir datos sensibles en logs.

- **Accesibilidad (frontend)**

Labels asociados, mensajes de error accesibles y navegación por teclado funcional.

Entregables

Entrega un repositorio en GitHub con la ruta elegida.

- **Estructura sugerida del repositorio**

Una estructura válida (ajustable) es:

```
app/
  README.md
  backend/      # Ruta Backend - FullStack
  frontend/     # Ruta Frontend - FullStack
  .github/workflows/ci.yml
```

- **README**
  - Ruta elegida y descripción de la solución.
  - Arquitectura (servicios, comunicación, decisiones y trade-offs).

- Setup local paso a paso (backend/frontend/db/mocks).
- Variables de entorno (incluir. env.example).
- Listado de endpoints y autenticación (cómo obtener y usar el token).
- Catálogos implementados y su fuente (DB o configuración) y cómo se consumen.
- ORM elegido y cómo inicializar la base de datos (schema/seed).
- Scripts disponibles.
- Testing: cómo ejecutar pruebas.

- **CI/CD**

Configurar un pipeline mínimo que ejecute:

- Install dependencies
- Lint
- Run tests
- Build

Criterios de aceptación

- **Generales**
  - Catálogos disponibles y consumidos/validados correctamente (según ruta).
  - Contratos REST consistentes y estables para quotes/policies/catalogs/auth.
  - Validación de inputs (requeridos, rangos y pertenencia a catálogos).
  - Manejo consistente de errores (formato único).
  - README reproducible y completo.
  - CI/CD en GitHub Actions ejecuta correctamente.

- **Ruta Frontend**
  - Formulario de cotización funcional con carga de catálogos y validaciones.
  - Visualización de prima y breakdown.
  - Flujo de emisión de póliza con autenticación simulada y UI clara.
  - Estados de carga y errores visibles (catálogos, cotización, emisión).
  - Tests mínimos en UI.

- **Ruta Backend**

- POST /quotes crea cotización y devuelve prima + breakdown; persiste el registro.
- GET /quotes/{id} retorna la cotización persistida.
- POST /policies emite póliza y persiste; requiere token (401/403 correctos).
- GET /policies/{id} retorna la póliza persistida; requiere token.
- POST /auth/login retorna token utilizable en Authorization: Bearer.
- Catálogos expuestos por API y validados en POST /quotes.
- ORM implementado; el postulante incluye schema y mecanismo de inicialización/seed.
- Swagger/OpenAPI disponible.
- Tests mínimos de API.

- **Ruta Fullstack**
  - Flujo completo UI -> API real: cotizar -> ver resultado -> emitir póliza.
  - No es posible emitir ni consultar pólizas sin autenticación.
  - Catálogos consumidos desde backend real.
  - Errores de API se reflejan en UI de forma consistente y comprensible.